Sales Reporting Dashboard

Database connection (join) notes - how the tables link together

Source: the captured raw SQL for **Sales > Reports** (*sales-reporting-dashboard-queries.md*), database **centric**, branch *feat/client-assignment*. Every join in all eight tabs, the filter bar and the lead drill-down drawer is catalogued here.

> **Read this first.** These notes are reconstructed from the *queries*, not from the schema. The join keys and join types are exactly what the SQL does. The cardinalities (1:1, N:1) are what the queries imply - they are not confirmed foreign keys. Section 8 gives you queries to verify them against the live database.

What is in this document

- **1. The big picture** - two relationship maps.
- **2. Table directory** - every table touched, its alias, and the columns the queries rely on.
- **3. Join catalog** - every join (36 entries, grouped A to E) with the ON condition, join type and purpose.
- **4. Join patterns** - the techniques the queries use (resolved status, pre-aggregation, LATERAL, and so on).
- **5. Tab-by-tab join map** - which tables each report touches.
- **6. Filter bar and drill-down** - how filters attach to the same spine.
- **7. Gotchas** - the traps that change numbers.
- **8. Verify it yourself** - SQL to confirm keys and cardinality.

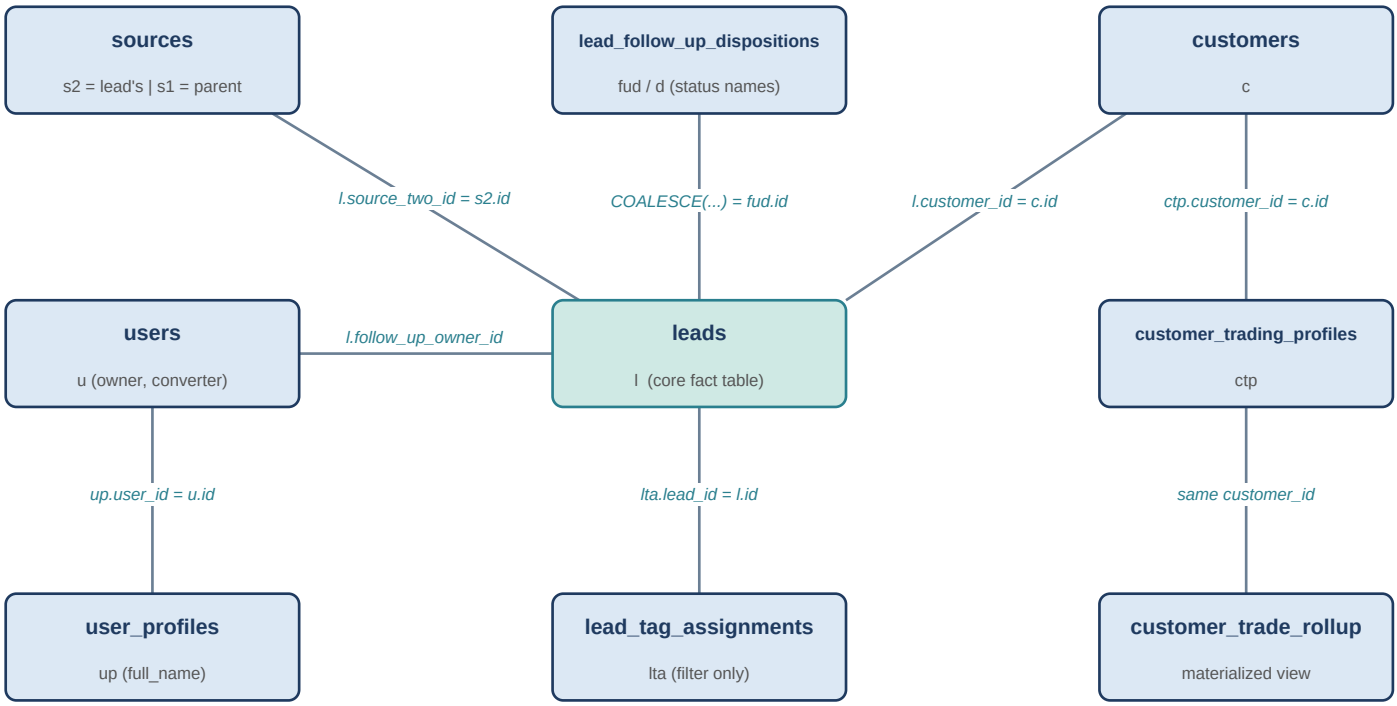
The one idea that explains most of the file

Almost every query is the same shape: start at **leads** (alias **l**), pin it to one project, one portal, not deleted, and one IST date window, then **LEFT JOIN** outward to descriptive tables (sources, users, customers, dispositions). Calls and trades are the exception: they are many-per-lead, so they are summarised in a CTE first and joined afterwards. Learn the leads spine and the six or seven recurring joins on it and you can read any tab.

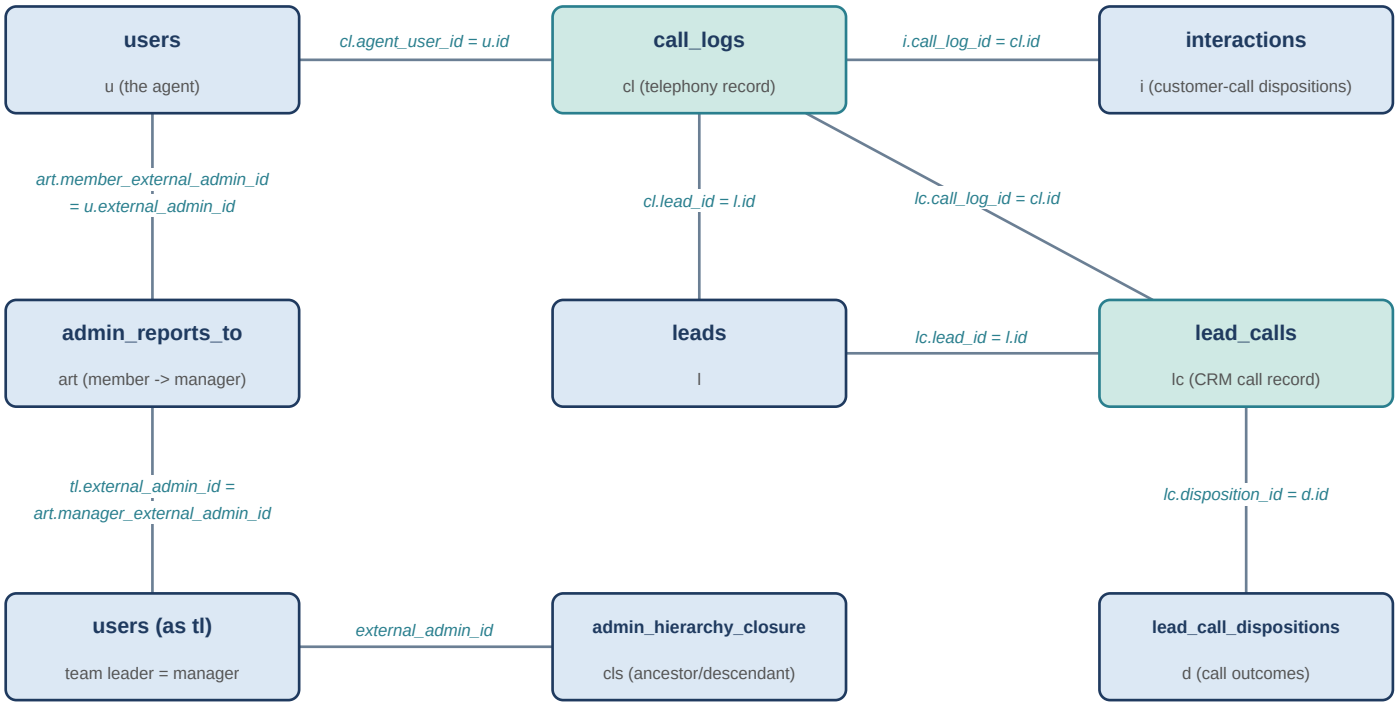
```
FROM leads l
  LEFT JOIN sources s2 ON s2.id = l.source_two_id      -- lead's source (level 2)
  LEFT JOIN sources s1 ON s1.id = s2.parent_id         -- its parent (level 1)
  LEFT JOIN users u ON u.id = l.follow_up_owner_id     -- agent who owns follow-up
  LEFT JOIN user_profiles up ON up.user_id = u.id      -- agent's full name
  LEFT JOIN customers c ON c.id = l.customer_id        -- the signed-up customer, if any
  LEFT JOIN customer_trading_profiles ctp ON ctp.customer_id = c.id
WHERE l.project_id = '<project>' AND l.portal_id = '<portal>'
  AND l.deleted_at IS NULL
  AND l.date_of_lead >= '<from IST>' AND l.date_of_lead < '<to IST exclusive>'
```

1. The big picture

Map A - the lead spine. Boxes are tables; the line label is the join condition. The green box is the hub every report starts from.



Map B - calls and the org hierarchy. There are **two call tables** on purpose: **call_logs** is the telephony record (status, answered_at, duration) and **lead_calls** is the CRM call record (disposition chosen by the agent). Agents are linked to team leaders through **external_admin_id**, not through users.id.



Not drawn but used: lead_sop_rules, lead_sop_excluded_owners, lead_sop_settings, portal_holidays and lead_callbacks (SOP and SLA tab - see section 3, group D); projects and portals (looked up once to get the two ids pinned in every query); customer_activity_events (the raw trade feed, deliberately *not* read - the rollup view is used instead).

2. Table directory

Aliases are reused with different meanings between queries, so always check the FROM clause. The same **users** table appears under many aliases because it plays different roles (owner, converter, agent, team leader, manager).

Table	Alias	What it holds	Columns the queries rely on
leads	l	One row per lead. The hub table.	id, project_id, portal_id, deleted_at, date_of_lead, created_at, source_two_id, follow_up_owner_id, converted_owner_id, follow_up_disposition_id, deposit_observed_at, customer_id, converted_at, call_count, reference_no, client_name, phone_calling_bidx, first_assigned_at
sources	s2, s1	Lead source tree. Same table joined twice (child and parent).	id, name, parent_id, portal_id
users	u, co, fo, tl, uo, ua, cbu	Staff / agents. Several aliases of the same table in different roles.	id, email, external_admin_id
user_profiles	up, fop, cop, tlp, cbp	Display names for users.	user_id, full_name
lead_follow_up_dispositions	fud, d	Follow-up status catalog per portal. Two rows are system statuses.	id, portal_id, name, system_key ('new_lead' / 'deposited'), color, active, sort_order, deleted_at
customers	c	Signed-up customer linked to a lead.	id, name, registered_at
customer_trading_profiles	ctp	Trading / deposit / referral facts per customer.	customer_id, zynixx_client_id, referrer_client_id, arc_client_id, first_trade_date, first_deposit_date, first_deposit_amount, total_deposit
customer_trade_rollup	r, t	Materialized view of trades per customer. Refreshed by a worker about every 5 minutes.	customer_id, trades, trade_value, first_leg, last_leg, refreshed_at
call_logs	cl	Telephony record of each call placed. Source of 'connected' and talk time.	id, portal_id, lead_id, customer_id, agent_user_id, status, started_at, answered_at, ended_at, duration_seconds
lead_calls	lc	CRM call record with the agent's disposition. Drives leads.call_count.	lead_id, call_log_id, disposition_id, created_at, id
lead_call_dispositions	d	Call outcome catalog per portal.	id, portal_id, name, active, sort_order, deleted_at
interactions	i	Interaction log; used for dispositions on customer (non-lead) calls.	call_log_id, disposition, dispositioned_at, created_at
lead_tag_assignments	lta	Lead-to-tag link. Used only in an EXISTS filter.	lead_id, tag_id
lead_callbacks	cb	Scheduled callbacks for a lead.	lead_id, due_at, created_by, completed_at, cancelled_at
admin_reports_to	art	Direct manager of each staff member.	member_external_admin_id, manager_external_admin_id
admin_hierarchy_closure	cls	Full org tree (every ancestor / descendant pair).	ancestor_external_id, descendant_external_id
lead_sop_settings	-	Per-portal SOP settings (business hours etc.). Read, then values inlined.	portal_id, business_start_minute, business_end_minute, count_working_days, default_base_calls, default_cadence_days
lead_sop_rules	r	Per-status call quota rules.	portal_id, disposition_id, exempt, day_one_only, late_arrival_nth_call, base_calls, cadence_days
lead_sop_excluded_owners	x	Agents excluded from SOP checks.	portal_id, user_id
portal_holidays	h	Holiday calendar per portal.	portal_id, holiday_date
projects, portals	-	Looked up once for the two ids pinned in every query.	id, name

3. Join catalog

Join type matters. LEFT JOIN keeps every lead even when the other side is missing (that is how Unsourced, Unassigned and not-signed-up rows survive). INNER JOIN, used for call tables against the cohort, deliberately drops rows that do not match. Placeholders in angle brackets stand for the pinned values.

A. Lead dimensions - the leads spine

Join (from to)	ON condition	Type	Purpose and notes
leads to sources (s2)	`s2.id = l.source_two_id`	LEFT, N:1	Lead's source (level 2). A lead with no source keeps its row.
sources s2 to sources s1	`s1.id = s2.parent_id`	LEFT, N:1	Self-join. Source 1 is the parent of source 2. A NULL s1 is shown as Unsourced.
leads to users (owner)	`u.id = l.follow_up_owner_id`	LEFT, N:1	Agent owning the follow-up. NULL owner means Unassigned.
leads to users (credit, tab 6)	`u.id = COALESCE(l.converted_owner_id, l.follow_up_owner_id)`	LEFT, N:1	Conversion credit: the converter if recorded, otherwise the follow-up owner.
leads to users (converter, tab 7)	`co.id = l.converted_owner_id`	LEFT, N:1	Shown as client owner in the Tag Status sheet.
users to user_profiles	`up.user_id = u.id`	LEFT, 1:1	Gets full_name. Labels use u.email; full_name is the tooltip.
leads to customers	`c.id = l.customer_id`	LEFT, N:1	Customer created from the lead. NULL customer_id means not signed up.
customers to customer_trading_profiles	`ctp.customer_id = c.id`	LEFT, 1:1 (expected)	Referral, deposit and client-id facts. Tab 7 joins on l.customer_id instead - same result.
leads to lead_follow_up_dispositions	`fud.id = COALESCE(CASE WHEN l.deposit_observed_at IS NOT NULL THEN <deposited id> END, l.follow_up_disposition_id, <new_lead id>)`	LEFT, N:1	Resolved status, not the raw column. See section 4.3.

B. Calls

Join (from to)	ON condition	Type	Purpose and notes
call_logs to cohort	`k.id = cl.lead_id`	INNER	Join used as a filter: keep only calls for leads in the window cohort. Counts all calls ever, not just in-window.
cohort to calls (summary CTE)	`c.lead_id = l.id`	LEFT, 1:1	Calls are pre-summarised to one row per lead, so no row multiplication.
lead_calls to cohort	`k.id = lc.lead_id`	INNER	Same filter-join idea, for lead_calls (tab 7). Gives last_call.
lead_calls to lead_call_dispositions	`d.id = lc.disposition_id`	INNER (in subquery)	Newest call's outcome: ORDER BY lc.created_at DESC, lc.id DESC LIMIT 1.
call_logs to users (agent, tab 8)	`u.id = cl.agent_user_id`	INNER	Drops any call with no agent. Note the other tabs use LEFT JOIN to users.
w (calls) to lead_calls	`lc.call_log_id = w.id`	LEFT	Brings the lead call's disposition onto each dialed call (tab 8b).
lead_calls to lead_call_dispositions (8b)	`d.id = lc.disposition_id`	LEFT	Disposition name for lead calls.
interactions to w (customer calls)	`w.id = i.call_log_id AND w.lead_id IS NULL`	INNER	Customer calls (no lead) get their disposition from interactions, newest one via DISTINCT ON.

C. Organisation hierarchy

Join (from to)	ON condition	Type	Purpose and notes
users to admin_reports_to	`art.member_external_admin_id = u.external_admin_id`	LEFT	Finds the agent's manager row. Key is external_admin_id, not id.
admin_reports_to to users (tl)	`tl.external_admin_id = art.manager_external_admin_id`	LEFT	Team leader. Second alias of users.
users (tl) to user_profiles	`tlp.user_id = tl.id`	LEFT	Team leader's full name.

Join (from to)	ON condition	Type	Purpose and notes
Filter: owner + their team	`uo JOIN cls ON cls.descendant_external_id = uo.external_admin_id; JOIN ua ON ua.external_admin_id = cls.ancestor_external_id`	INNER (in EXISTS)	Walks the closure table: owner (uo) up to every ancestor (ua) so a manager's pick includes their whole team.

D. SOP and SLA (tab 4)

Join (from to)	ON condition	Type	Purpose and notes
cohort to lead_sop_rules	`r.portal_id = '<portal>' AND r.disposition_id = l.status_id`	LEFT	Per-status rule. No row means defaults from lead_sop_settings apply.
cohort to lead_sop_excluded_owners	`x.portal_id = '<portal>' AND x.user_id = l.follow_up_owner_id`	LEFT	Non-NULL x.user_id means the owner is excluded (required calls = 0).
cohort to call_stats	`cs.lead_id = l.id`	LEFT, 1:1	Total, first and last call times per lead.
cohort to call_ranks (nth)	`nth.lead_id = l.id AND nth.rn = r.late_arrival_nth_call`	LEFT	Picks the Nth call (row_number). If the rule has no nth call it never matches.
cohort to callback	`cb.lead_id = l.id`	LEFT, 1:1	Next open callback (DISTINCT ON lead, earliest due_at).
callback to users / user_profiles	`cbu.id = cb.created_by; cbp.user_id = cbu.id`	LEFT	Who set the callback.
cohort CROSS JOIN cal_end	`(none)`	CROSS	cal_end is one row (working days up to today), attached to every lead.
cohort to calendar (LATERAL)	`c.day = l.sop_start_day`	LEFT LATERAL	Computes days elapsed = cal_end.wd - c.wd + (1 if start day is working).
calendar to portal_holidays	`h.portal_id = '<portal>' AND h.holiday_date = d::date`	LEFT	Marks holidays as non-working days.

E. Trading data (tab 7) and filter-bar semi-joins

Join (from to)	ON condition	Type	Purpose and notes
customer_trade_rollup to cohort customers	`k.customer_id = r.customer_id (k = SELECT DISTINCT customer_id FROM cohort WHERE customer_id IS NOT NULL)`	INNER	Restricts the materialized view to this cohort's customers before use.
facts to trades	`t.customer_id = l.customer_id`	LEFT	Trade count, value, first/last trade.
facts to customer_trading_profiles	`ctp.customer_id = l.customer_id`	LEFT	Deposit and client-id facts.
facts to calls / connected	`ca.lead_id = l.id; cn.lead_id = l.id`	LEFT, 1:1	last_call from lead_calls; connected_count from call_logs. Both pre-aggregated.
Filter: tag	`EXISTS (... lta.lead_id = l.id AND lta.tag_id = ANY(...))`	EXISTS	Semi-join: lead qualifies if at least one matching tag exists, no duplicates.
Filter: source 1	`l.source_two_id IN (SELECT s.id FROM sources s WHERE s.parent_id = ANY(...) AND s.portal_id = '<portal>')`	IN subquery	Selecting a Source 1 means any child source 2 of it.

4. Join patterns used in these queries

The same handful of techniques is reused everywhere. Recognise them and the long queries become short.

4.1 The leads spine

Always LEFT JOIN outward from leads. The leading filters (project, portal, deleted_at, date window) sit in WHERE on l. The window is half-open: start inclusive, end exclusive, both as IST midnights with +05:30.

4.2 Self-join on sources

One table, two roles. The lead points at a level-2 source; that row points at its parent.

```
LEFT JOIN sources s2 ON s2.id = l.source_two_id
LEFT JOIN sources s1 ON s1.id = s2.parent_id   -- s1 is NULL for Unsourced
```

4.3 Resolved follow-up status (join on an expression)

The raw column **follow_up_disposition_id** is not the status you see. A deposited lead is forced to the system 'deposited' status, and a lead with no disposition is the system 'new_lead' status. The first non-NULL of the three wins:

```
fud.id = COALESCE(
  CASE WHEN l.deposit_observed_at IS NOT NULL THEN <id of system_key 'deposited'> END,  -- 1st: deposited
  l.follow_up_disposition_id,                                                           -- 2nd: the agent's choice
  <id of system_key 'new_lead'>                                                         -- 3rd: default
)
```

The two system ids are inlined as scalar subqueries on *lead_follow_up_dispositions(portal_id, system_key)*, so only the portal id needs changing. Any filter on status must use this same expression, never the raw column.

4.4 Summarise first, then join (avoiding row multiplication)

A lead can have many calls. Joining calls straight onto leads and then counting leads would multiply each lead by its calls. The queries therefore roll calls up to **one row per lead** in a CTE and LEFT JOIN that:

```
WITH cohort AS (SELECT l.id, ... FROM leads l ... WHERE <window>),
     calls  AS (SELECT cl.lead_id, count(*) FILTER (...) AS calls, ...
                  FROM call_logs cl JOIN cohort k ON k.id = cl.lead_id
                 GROUP BY 1)
SELECT ... FROM cohort l LEFT JOIN calls c ON c.lead_id = l.id GROUP BY ...
```

4.5 JOIN used as a filter

JOIN cohort k ON k.id = cl.lead_id contains no selected columns from k. Its only job is to keep calls whose lead is in the window. That is also why connectivity counts every call ever placed to a lead that arrived in the window.

4.6 Constants inside ON (config tables)

Settings tables hold one set of rows per portal. The portal is pinned inside the ON clause so the LEFT JOIN cannot pick up another portal's rule:

```
LEFT JOIN lead_sop_rules r
       ON r.portal_id = '<portal>' AND r.disposition_id = l.status_id
```

4.7 LATERAL and CROSS JOIN for working-day arithmetic (tab 4)

A **calendar** CTE numbers each day with a running count of working days (*sum(...) OVER (ORDER BY d)* = wd), skipping weekends and portal holidays. **cal_end** is today's row. A LATERAL subquery then looks up each lead's start-day row and subtracts, giving days elapsed for that lead. CROSS JOIN is safe here because cal_end is exactly one row.

```
CROSS JOIN cal_end
LEFT JOIN LATERAL (
  SELECT (cal_end.wd - c.wd + CASE WHEN c.is_working THEN 1 ELSE 0 END) AS elapsed
    FROM calendar c WHERE c.day = l.sop_start_day
) days ON true
```

4.8 One row per group with DISTINCT ON

Used to pick the earliest open callback per lead (ORDER BY lead_id, due_at) and the newest disposition per customer call (ORDER BY call_log_id, dispositioned_at DESC). It guarantees one row per key, so the following LEFT JOIN cannot duplicate.

4.9 Ranked rows: row_number() then join on the rank

call_ranks numbers each lead's calls by started_at. The SOP rule says which call number matters (*late_arrival_nth_call*), and the join *nth.rn = r.late_arrival_nth_call* fetches exactly that call's timestamp.

4.10 EXISTS and IN instead of joins for filters

Tag and team filters use EXISTS / IN subqueries. They test membership without multiplying lead rows - a lead with three matching tags still counts once.

4.11 Two arms, UNION ALL (tab 2)

Not a join, but it explains the conversion numbers. The **leads arm** groups by date_of_lead (needs customers and ctp); the **funded arm** groups by converted_at (needs neither). They are stacked with UNION ALL and summed, so one date row carries both clocks.

4.12 ROLLUP for non-additive totals (tab 8c)

Unique dials per team leader cannot be added up from agent rows (the same lead may be dialled by two agents). *GROUP BY ROLLUP(tl.id)* recomputes DISTINCT counts per leader and for the grand total; GROUPING(tl.id) = 1 marks the total row.

5. Tab-by-tab join map

Tab / query	Base table	Tables joined (and how)
1a Status catalog	`lead_follow_up_dispositions`	No joins. Filter portal_id, order by sort_order.
1b Status x Source 1	`leads l`	LEFT sources s2, LEFT sources s1; status via COALESCE subqueries on dispositions.
1c Connectivity by source	`cohort (leads)`	cohort uses s2, s1. calls CTE = call_logs INNER cohort. Final: cohort LEFT calls, LEFT s2, LEFT s1.
1d Connectivity by agent	`cohort (leads)`	Same as 1c, then LEFT users (owner), LEFT user_profiles.
2a Daily conversion	`leads l (two arms)`	Leads arm: LEFT customers, LEFT customer_trading_profiles. Funded arm: leads only.
2b Conversion x Source 1	`leads l (two arms)`	Leads arm: customers, ctp, s2, s1. Funded arm: s2, s1.
3 Lead Flow	`leads l`	LEFT s2, LEFT s1. Hour and market-hours maths on date_of_lead.
4b SOP facts	`cohort (leads)`	s2, s1, users, user_profiles, dispositions, lead_sop_rules, lead_sop_excluded_owners, call_stats, call_ranks, lead_callbacks (+ users, profiles), CROSS cal_end, LATERAL calendar (portal_holidays).
5 Agent status	`leads l`	LEFT s2, s1, users, user_profiles. Same pivot as 1b by owner.
6 Conversions by agent	`leads l`	LEFT s2, s1, users (COALESCE credit), user_profiles, customers, customer_trading_profiles.
7 Tag Status (a-d)	`cohort (leads)`	CTEs: lead_calls (last call), call_logs (connected), customer_trade_rollup. facts: LEFT ctp, calls, connected, trades, users x2, user_profiles x2; subquery lead_calls + lead_call_dispositions for call_outcome. Final: LEFT dispositions.
7e Lookups	`customer_trade_rollup / lead_call_dispositions`	No joins.
8a Dialer per agent	`call_logs cl`	INNER users (agent), LEFT user_profiles, LEFT admin_reports_to, LEFT users (tl), LEFT user_profiles (tl).
8b Disposition split	`call_logs (w) + interactions (ci)`	w = 8a joins. ci = interactions INNER w (customer calls). Final: w LEFT lead_calls, LEFT lead_call_dispositions, LEFT ci.
8c Unique dials	`call_logs cl`	Same joins as 8a with GROUP BY ROLLUP(tl.id).
8d Disposition catalog	`lead_call_dispositions`	No joins.
Drill-down drawer	`leads l`	LEFT s2, s1, customers, ctp, users, user_profiles, dispositions (COALESCE status); correlated subquery on lead_calls for lastCallAt.

What to notice

- Tabs 1b, 3, 5 and 6 are 'leads plus descriptive lookups' - the simplest shape.
- Tabs 1c/1d, 4 and 7 are 'cohort + per-lead summaries' - the CTE-then-join shape.
- Tab 8 is the only one that starts from **call_logs** instead of leads, so its window is call time (started_at), not lead arrival.
- Tabs 2 and 6 use two time clocks (date_of_lead and converted_at) in one query.

6. Filter bar and drill-down drawer

Filters never add new row-multiplying joins. They are WHERE conditions (often EXISTS / IN) on the same **l** row, which is why the filtered report total always matches the drawer count.

Filter bar item	How it attaches to the spine
Source 1	`l.source_two_id IN (SELECT s.id FROM sources s WHERE s.parent_id = ANY(...) AND s.portal_id = '<portal>')`
Source 2	`l.source_two_id = ANY(ARRAY['<src2>']::uuid[])`
Unassigned only	`l.follow_up_owner_id IS NULL` (overrides both owner pickers)
Follow-up owner	`l.follow_up_owner_id = ANY(...)`
Owner + their team	`EXISTS (SELECT 1 FROM users uo JOIN admin_hierarchy_closure cls ON cls.descendant_external_id = uo.external_admin_id JOIN users ua ON ua.external_admin_id = cls.ancestor_external_id WHERE uo.id = l.follow_up_owner_id AND ua.id = ANY(...))`
Both owner pickers	The two clauses above are OR'd together.
Follow-up status	The resolved COALESCE expression from 4.3, `= ANY(...)`
Tag	`EXISTS (SELECT 1 FROM lead_tag_assignments lta WHERE lta.lead_id = l.id AND lta.tag_id = ANY(...))`
Dialer filters	Apply to call_logs: `cl.lead_id IS NULL / IS NOT NULL` (call type), `tl.id = '<user>'` (team leader), `cl.agent_user_id = ANY(...)` (agent).

Drill-down drawer

Clicking any number runs two queries over the same predicate: a page query (25 rows, adds LEFT joins for display columns: customers, ctp, users, user_profiles, resolved status, last call) and a lean count query that joins only what the predicate needs (sources, and customers when the bucket is full / half sign-up). Because both share the predicate, the drawer total equals the number you clicked.

Two joins worth remembering in the drawer

- `(SELECT MAX(lc.created_at) FROM lead_calls lc WHERE lc.lead_id = l.id)` - last call comes from **lead_calls**, not call_logs.
- If a bucket predicate uses `c.registered_at` (full / half sign-up), the count query must also include `LEFT JOIN customers c ON c.id = l.customer_id`, otherwise it errors.

7. Gotchas that change the numbers

#	Gotcha	Detail
1	**Two call tables, different meanings.**	call_logs = what the dialer placed (answered_at, duration). lead_calls = what the agent logged (disposition). leads.call_count comes from lead_calls. Connected can only come from call_logs. Migrated Zoho history has lead_calls but no call_logs.
2	**Always dialled calls only.**	Both tabs 1 and 8 exclude status 'placing' and 'call_failed'. Forget this and call counts inflate.
3	**Connectivity uses the lead's arrival window, not the call's.**	1c/1d count every call ever placed to leads that arrived in the window. The Dialer (tab 8) counts calls placed in the window to anyone. The two will not match.
4	**Raw status is not the status.**	Use the COALESCE expression. A lead with deposit_observed_at set always reads Deposited, whatever follow_up_disposition_id says.
5	**Date column: date_of_lead, never created_at.**	created_at is the insert time (import day for imported books). Conversion tabs also bucket on converted_at, so a column can fall outside the window end.
6	**IST everywhere.**	Buckets use AT TIME ZONE 'Asia/Kolkata'. The window end is exclusive: IST midnight of the day after the last day.
7	**Soft deletes.**	leads.deleted_at IS NULL is always applied. The disposition catalogs also carry deleted_at: tab 1a returns deleted rows, 8d filters them out.
8	**Portal scope on every config table.**	Sources, dispositions, SOP rules, excluded owners and holidays are per portal. leads is filtered by project and portal; config tables by portal (as a constant in ON or WHERE).
9	**NULL means a real row, not an error.**	NULL s1 is Unsourced. NULL owner is Unassigned. NULL customer_id is not signed up. Pivots keep these rows (Unsourced pinned last).
10	**INNER vs LEFT on users.**	Tab 8 uses INNER JOIN users, so a call with no agent_user_id disappears. Lead tabs use LEFT JOIN users, so unassigned leads stay.
11	**Hierarchy joins go through external_admin_id.**	Team leader and closure joins link users on external_admin_id. Everywhere else users link on users.id. Mixing them up returns no rows.
12	**Alias reuse.**	c is customers in most queries, but is the calls CTE in 1c and the calendar in the SOP LATERAL. d is dispositions in some queries and the daily date in others. Read the FROM clause every time.
13	**Trades are up to 5 minutes stale.**	customer_trade_rollup is a materialized view. The 'Trade data refreshed at' label comes from max(refreshed_at).
14	**Totals that cannot be summed.**	Unique dials per team leader (8c) and any DISTINCT count must be recomputed, not added from child rows.
15	**The SQL is not the whole report.**	Percentages, pivoting, week/month folding and the SOP/SLA verdicts (met / short / exempt) are done in TypeScript after the query.

Possible row-multiplication risks to check (not stated in the source). (1) If a customer can have more than one customer_trading_profiles row, every lead of that customer duplicates in tabs 2, 6 and 7. (2) If a staff member appears under more than one manager in admin_reports_to, that agent's dialer calls would be counted once per manager in tab 8. Section 8 has the queries to test both.

8. Verify it yourself

These are standard PostgreSQL catalog queries plus two cardinality checks. They confirm the relationships in this document against the real database.

8.1 List the real foreign keys for the tables in this document

```
SELECT conrelid::regclass AS table_name, conname,
       pg_get_constraintdef(oid) AS definition
  FROM pg_constraint
 WHERE contype = 'f'
   AND conrelid::regclass::text IN (
     'leads','sources','users','user_profiles','customers',
     'customer_trading_profiles','call_logs','lead_calls',
     'lead_call_dispositions','lead_follow_up_dispositions',
     'lead_callbacks','lead_tag_assignments','lead_sop_rules',
     'lead_sop_excluded_owners','portal_holidays','interactions')
 ORDER BY 1, 2;
```

If a relationship in this document has no matching foreign key, the application enforces it (or it is just a convention) - worth knowing before you trust it.

8.2 Inspect one table in psql

```
\d+ leads
\d+ call_logs
\d+ lead_calls
```

8.3 Does a customer have more than one trading profile?

```
SELECT customer_id, count(*) FROM customer_trading_profiles
 GROUP BY 1 HAVING count(*) > 1 LIMIT 20;      -- expect zero rows
```

8.4 Does an agent have more than one manager row?

```
SELECT member_external_admin_id, count(*) FROM admin_reports_to
 GROUP BY 1 HAVING count(*) > 1 LIMIT 20;      -- expect zero rows
```

8.5 Do call_logs and lead_calls agree on connected leads?

```
SELECT
  (SELECT count(DISTINCT lead_id) FROM call_logs WHERE answered_at IS NOT NULL) AS leads_connected_per_call_logs,
  (SELECT count(DISTINCT lead_id) FROM lead_calls)                              AS leads_with_any_lead_call;
```

8.6 How many leads have each kind of missing link?

```
SELECT count(*)                                    AS leads,
       count(*) FILTER (WHERE source_two_id IS NULL)        AS no_source,
       count(*) FILTER (WHERE follow_up_owner_id IS NULL)   AS unassigned,
       count(*) FILTER (WHERE customer_id IS NULL)          AS not_signed_up,
       count(*) FILTER (WHERE converted_at IS NOT NULL)     AS converted
  FROM leads
 WHERE portal_id = '9d49d719-12e6-489d-89db-c55d325bdfad' AND deleted_at IS NULL;
```

Quick reference - the five join keys to memorise

Relationship	Key
Lead to source, source to parent	`l.source_two_id = s2.id; s2.parent_id = s1.id`
Lead to agent	`l.follow_up_owner_id = u.id (and converted_owner_id)`
Lead to customer to trading facts	`l.customer_id = c.id; ctp.customer_id = c.id`
Lead to calls	`call_logs.lead_id = l.id; lead_calls.lead_id = l.id; lead_calls.call_log_id = call_logs.id`
Agent to team leader	`users.external_admin_id = art.member_external_admin_id; art.manager_external_admin_id = tl.external_admin_id`